

“You want to do what?” Breaking the Rules to Increase Customer Satisfaction

Mike Prior
SumTotal Systems Inc.
Des Moines, IA USA
mprior@sumtotalsystems.com

Abstract—Customer satisfaction. Every customer support organization strives to increase and maintain a higher level of customer satisfaction. This is a story of how one software organization attempted to increase customer satisfaction by increasing predictability and response times to reported defects. Along the way they were able to establish trust with the customer support organization by implementing a process that seemed counterintuitive to its stakeholders.

Keywords—Agile development, work in progress, cycle times, backlogs, queues, software defect management

I. INTRODUCTION

SumTotal Des Moines is a Learning Management System (LMS) provider that uses a Software as a Service (SaaS) approach to delivering its product to customers. The Des Moines operation consists of about 180 employees, including 20 developers on the IT team and 25 customer support personnel. Release iterations are one month long. Each release includes defect fixes as well as new features. The development team is made up of two groups: the feature development team and the production support team.

There are currently seven developers on the production support team, but the number of developers can fluctuate between five and seven depending on the demands of other projects. All developers, regardless of their association with either the production support (PS) team or the feature team, employ test driven development (TDD) and acceptance test driven development (ATTD) techniques.

Additionally, at the time this author was hired as the PS team manager, all developers were expected to pair program. The PS team's main responsibility is to work defects reported by customers that are created by the customer support (CS) team. The PS team and its interaction with the customer support team will be the primary focus of this case study.

II. THE ENVIRONMENT BEFORE WE MADE CHANGES

When this author was hired the PS team and the CS team had an already agreed upon process for working customer reported defects. This process was as follows:

- 1) Customer calls with issue.
- 2) Customer support representative creates defect report ticket in ticketing system.

- 3) Defect number is placed in a primary queue where the customer support representative feels is appropriate compared to existing defects in the queue (backlog).
- 4) Developers in the production support group pick from the top of that queue.
- 5) Developers do an initial “triage” of the defect.
- 6) Developer assigns the defect to one of two other queues based on the level of effort to fix

Despite the attempt to give each defect immediate attention, with this process it seemed as though customers were becoming more and more upset that their defects were not getting the attention they deserved. Sometimes defects could remain in any one of the queues for upwards of 120 days. This led to customers escalating their defects to get them worked right away. However, by doing this it only increased the time to delivery of all the other defects that were now waiting in a queue. Subsequently, this led to those slighted customers escalating their defects, the PS team working them, and other defects being pushed down the list. This cycle was creating upset customers.

III. QUEUES

In addition to creating upset customers, this process did not lend to efficient use of expensive PS team developer time. Several queues were used to segment the work based on level of effort. Once a developer did an initial “triage” of the defect in the main queue they would place the defect in either the “to be worked” queue, meaning it was a low level of effort, or they would place the defect in the “needs prioritization” queue, meaning it was a high level of effort and needed integration into the product management backlog (Fig. 1).

Paradoxically, these queues were initially implemented with the sole purpose of trying to increase responsiveness to the customer. The thought was that after the initial triage of the defect, the CS team could then tell the customer that their defect has been “looked at” and was either going to be fixed in the next few releases or it required a high level of effort to fix and needed to be worked into the product roadmap. Either way the customer was still waiting.

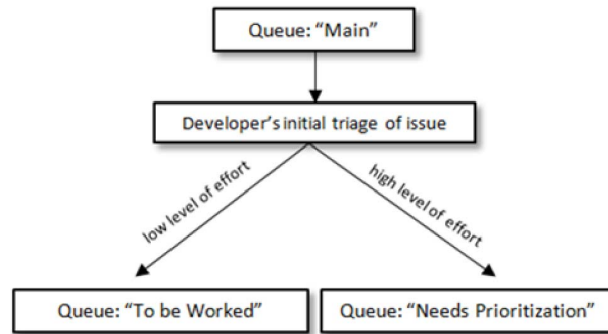


Figure 1. Flow of defects between queues

IV. CHALLENGES OF THE OLD SYSTEM

Challenges of the old system presented us with opportunities for improvement. As mentioned earlier, the old process began to create more upset customers as escalated defects became the norm. While the goal of segmenting queues was to increase customer satisfaction through quicker response times, this ended up having the opposite effect as the throughput of fixes to production was hindered by the time spent “being responsive”. Customers wanted results, not just management of defects.

Another challenge of the old system was the fact that a defect would be touched twice by a developer before getting an actual fix. This seemed an obvious area for improvement. Perhaps the biggest challenge we faced with the old process was gaining the trust of the CS team and customers. When more defects were coming into the PS team than were going out, the first response was to focus on our engineering practices. Statements like, “if you don’t pair, won’t you get twice as much done?” and, “can we take resources off of feature development to help with fixing customer defects?” were heard weekly.

V. THE TRANSITION

During the late summer of 2010, organizational changes were made such that the CS team was to be part of the IT branch of the organization, rather than under the Services branch. With this change in structure, a new Director of Customer Support was hired. This change, and the perception of constantly upset customers due to long lead times, presented the organization with the opportunity to change the defect resolution process. Over a few days, meetings were held with the CS team to brainstorm ideas for change. The first thing we did was agree on a goal, and that goal was unanimously to get customers their fixes much faster than we had been in the past.

The PS team was able to expose waste by creating, with the help of the CS team, a value stream map of the old process. The PS team diagramed the system-wide process for getting a customer defect worked which included the

amount of time a defect spent in each bucket and how many resources were involved in getting it to its goal. When all was said and done we found that the actual working time of a defect throughout the system was roughly 4 days. The time the defect spent waiting in various queues to be worked was roughly 100 days, and each day our backlog was growing. If the goal was to get customers fixes quickly, these waiting times were preventing us from reaching our goal. So we all could agree that a change to the process was necessary.

VI. MAKING CHANGES

Before we made any process changes, the CS team was able to prioritize defects in the main queue above others that may have been there for a long time. This resulted in upset calls from customers which ultimately ended up in an escalated defect. The first change we made was to consolidate the three queues into one main queue. No longer would a developer pick a defect from the main queue, troubleshoot to find the cause, then pass it to another queue (either “to be worked” or “needs prioritization”) to be picked up and fixed later.

This new main queue was a first-in-first-out (FIFO) queue where the developer would pick the oldest defect and work it until it was fixed and ready for release. This meant developers would touch a defect only once, allowing them to retain the context and eliminate waste. This also meant that since there was no more re-prioritizing, each defect that came in was assigned a place in line. So the PS team was able to start fixing defects with some predictability. The CS team agreed that establishing some sort of predictability of when a defect would be fixed would help in setting the customers expectation on the initial call and would help eliminate the overhead of daily updates to the customer with no new information to give them.

The second big change made was purely for the benefit of our PS team. We established a process to keep track of each defect as it flowed through our development process. We created a big board (Fig. 2) with categories that matched our process. Those categories are:

- 1) Backlog
- 2) In Progress
- 3) Blocked
- 4) In QA
- 5) Release Ready
- 6) In Production

Each time a new defect was picked from the new FIFO queue, a developer pair was responsible for creating a card and placing it on the board within the In Progress column. As defects flowed out of In Progress into QA, the pair would move the card. Each day after stand up the team lead would take count of the number of defects in each category and mark them in a spreadsheet. This was the start of a process to not only establish some sort of predictability, but to also allow the PS team to determine some efficiencies such as the ability to generate the throughput rate, cycle and lead time. Most recently the PS team has started tying these key metrics to resourcing of the team and cost per defect. We are still working on how to adjust resources based on the metrics we are collecting.

VII. OVERCOMING CS TEAM CHALLENGES

Although a consensus was reached that the old process was creating upset customers and was also counterproductive to the goal of delivering fixes faster, there were still some concerns from the CS team about our proposed new process. They were concerned that responsiveness would suffer greatly since a defect could potentially sit in the single FIFO queue for a long time before there was any update for the customer. At least in the old process they were able to tell the customer something soon after the defect was created. In fact, the main reason that the CS team wasn't fully on board with the use of a single FIFO queue was for this very reason. The PS team countered that in the old process, even though the CS team was getting back to the customer relatively quickly, they were not providing any real value to the customer by telling them that someone had looked at their defect and that either it would be fixed within 120 days or that it was a more involved fix that would need to be prioritized by product management to squeeze in with an As-yet-to-be-determined feature enhancement. Moreover, it seemed more responsive to be able to tell a customer when they call and report a defect that their defect "would be fixed in X release", or, "looked at in X days" when before they really couldn't tell the customer when their defect would be fixed.

Another challenge the CS team faced was what to do with upset customers' defects that need looked at right away; those that could not wait in a queue. In every process there is going to be a need to expedite, and our process is no different. The PS team proposed that we create three expedite slots where the CS team could place one defect at a time in each slot and those would be the next defects to

be picked up and worked by a developer. That would mean three pair would focus on the tickets in the expedite slots while two pair could continue working FIFO queue defect in order to keep the backlog down. Visually, the expedite slots are represented by swim lanes across the top of our board with room only for one defects at time in each category (In Process, In QA, etc.). Additionally, the CS team lead is the only person that can expedite a defect to the PS team lead. This cuts down on confusion and forces the CS team lead to focus on the expedited defect.

VIII. MANAGING DEFECT FLOW

It is also important to note that we identified the PS team as being the bottleneck in this process. The PS team impressed upon the CS team that it was important to have the PS team working defects that absolutely needed to be fixed, or that have been vetted as a true defect. We didn't want developers spending time on defects that were not true defects, such as customer confusion around intended functionality, or that were not considered high priority.

Approximately three months after making the first change, the PS team, CS team and product management decided to hold weekly meetings to cover new defects reported the past week and decide if they should remain in the backlog for the PS team to fix. On average these meetings would result in about three to four defects being pulled out of the backlog. Even though our lead times were dropping, there were defects in the queue that were there 80 days before they were picked up by the PS team. Sometimes the developer would pick the next defect in the list, troubleshoot it, only to find out that they could not recreate the issue as it may have been fixed by another code drop or a data correction. This was a waste of developers' time so we asked the CS team to have a representative go through the next ten defects in the backlog and validate that each was still an issue. If a defect was not reproducible, the CS team representative would remove that defect from the queue and contact the customer. This assured the PS team that they were working only on issues that were relevant. This last change was made after four months into our process improvement.

IX. METRICS

We agreed that the most important metric is current lead time; e.g. how long until my defect is fixed. To fulfill our goal of giving predictability to the CS team, we needed to better understand how many defects were coming in and how many defects were being resolved. So each day the team lead would count the number of defects in six categories: Backlog, In Progress, Blocked, In QA, Release Ready, and In Production. These numbers helped us determine our cycle times and lead times for defects while our cumulative flow diagram (CFD) helped us visualize our defect flow (Fig. 3).

In the spirit of gaining trust and increasing transparency,

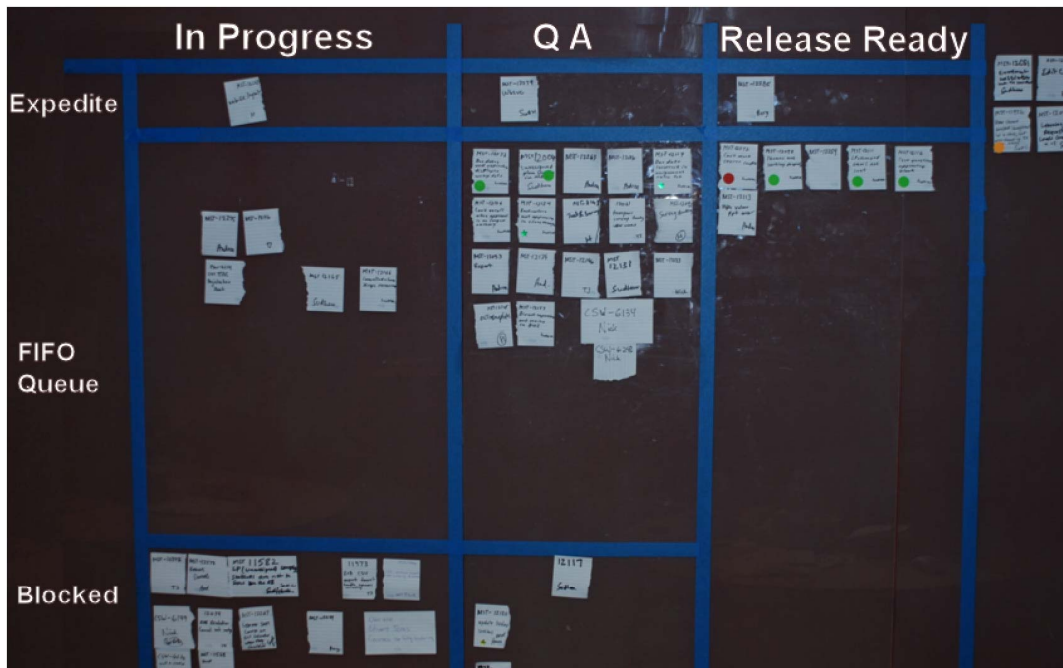


Figure 2. PS Team Work-in-Progress Board

the PS team created a dashboard that shows lead time, development cycle time, items in the backlog, and backlog queue time so that the CS team has a snapshot of the work in progress at any time. At the time of this writing the PS team has been keeping these metrics for eight months.

More recently the PS team has started classifying the types of defects that are reported. For example, if a developer looks at a defect, fixes it, then he/she will update the defect with a classification of what caused the defect. Was it caused by refactoring? Caused by the introduction of a new feature? Or was it in some legacy code that doesn't have any tests? The code base was written for our main product before we started using any test driven design (TDD) and thus there are still parts of the code base that do not have tests. Tracking this information will help us understand the type of work we are doing and help us to predict future trends. Right now, we are just updating the card on the board with a color-coded sticker to reflect the defect's category (Table 1). At the time of this writing, the PS team has been tracking defect classification data for three months.

X. RESULTS

In eight months we have gone from lead times of roughly 120 days to (at the time of this writing) 38 days with cycle times (the time it takes the defect to work its way through development and QA) of nine days. The CS team is now able to give a ballpark answer to the question, "how long before my defect is fixed?" on the initial call with the customer. With the PS team able to deliver more defect fixes faster

Table I
DEFECT CRITERIA CLASSIFICATION

Category	Criteria
Legacy Code	Greater than 8 months old/no tests
New Feature Defect	Defect originating from a new feature
Stepback – Feature	Was working, now it's not as a result of coding for a new feature
Stepback – PS Team	Was working, now it's not as a result of fixing another defect
Stepback – Refactoring	Was working, now it's not as a result of code refactoring
Data Migration	Data inconsistency due to a data migration
Unknown	

due to eliminating waste, the number of customer-escalated issues has decreased. This has allowed PS team to reduce the number of expedited slots from three to one. By doing this we are now able to put even more resources on to fixing the defects in the main FIFO queue with the goal of decreasing our lead time further and improving our system's stability.

XI. NEXT STEPS AND LESSONS LEARNED

Even though the lead time has decreased considerably, the PS team is still delivering defect fixes outside of one release iteration (currently within two releases). Our eventual goal is to get our lead time to within one release cycle. With the metrics the PS team has been keeping, it is now possible to start answering the question of, "what is a good resource allocation to the PS team in order to keep a lead time of one release out?" or, "what will be the change in throughput if

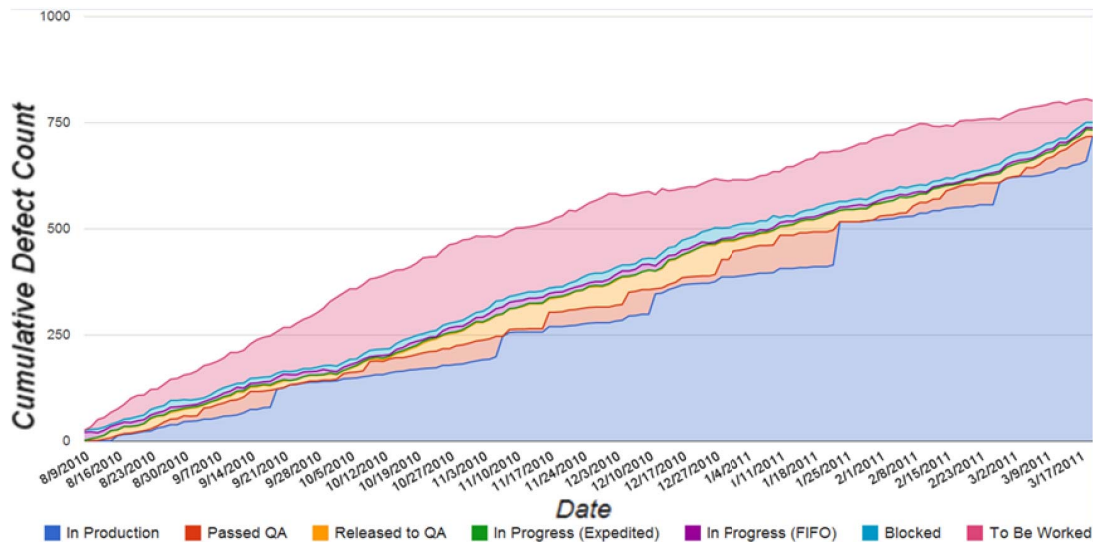


Figure 3. Cumulative Flow Diagram for Defect Tickets

a developer is added or taken away from the PS team?” or, “how much does a defect cost us to fix once it has been deployed to production?”

Although there are certain situations where you definitely need to give defects more attention than others, one thing this author would have done differently, given perfect hindsight, was to push harder to get the number of allowable expedite slots to our current minimum of one. With only one expedite slot available, this not only allows the development team to keep working on defects in the queue to maintain their flow, but it forces the CS team to reassess what they feel are the true “line stoppers”. It was a calculated decision not to press the CS team on lowering that from the start of our process improvement efforts as they were dealing with more upset customers than a system with only one expedite slot could support. Once we started delivering on our promises and predictions, it was easier convince the CS team to reduce the number of expedite slots to one.

This author would also consider not always having the developer’s within the PS team pair. Although there are times pairing is appropriate to solve complex problems or transfer knowledge, the nature of the defects that are coming in the PS team are mostly one off code/data fixes that don’t require too much interpretation of missed requirements. Currently, it is not a requirement to pair; we have left it to the discretion of the developers.

XII. CONCLUSION

Each organization has different goals, different experiences and different change agents, therefore there is no single best answer on how to create a stable defect resolution process. However, we at SumTotal were able to reduce our lead times to nearly one release out (previously about 4

releases out) and reduce the number of expedited tickets to its minimum (previously had up to 14 defects waiting to be expedited). We did this through incremental change to parts of our process that we identified as having a direct effect on customer satisfaction. Making these changes incrementally not only helped us assess their effects and adjust as necessary, but it also allowed our customer support organization to feel more comfortable with our proposed changes.

In the past engineering and support were siloed to the point of distrust due to a lack of transparency, so it was imperative that we break down any real or perceived “walls”. Agreeing on a common goal was a critical first step. This allowed both teams to move in the same direction and assured the CS team that engineering had a focus of customer satisfaction. Additionally, establishing visibility into the development processes for fixing defects was essential to gaining trust. We did this by inviting a representative from the CS Team to our stand-ups and by creating a dashboard that showed current lead times and development cycle times.

XIII. ACKNOWLEDGMENT

Many thanks to my colleagues Brandon Carlson and Tim Gifford for their leadership and mentoring through this process. Additionally, thanks to my team for taking ownership of this process and for holding each other accountable; true professionals at work. Thanks also to Rebecca Wirfs-Brock for her patience and insights during the writing of this paper.